

New Architecture is here

October 23, 2024 · 24 min read



The React Team

@reactjs / @reactnative



React Native 0.76 with the New Architecture by default is now available on npm!

In the [0.76 release blog post](#), we shared a list of significant changes included in this version. In this post, we provide an overview of the New Architecture and how it shapes the future of React Native.

The New Architecture adds full support for modern React features, including [Suspense](#), [Transitions](#), [automatic batching](#), and [useLayoutEffect](#). The New Architecture also includes new [Native Module](#) and [Native Component](#) systems that let you write type-safe code with direct access to native interfaces without a bridge.

This release is the result of a ground-up rewrite of React Native we've been working on since 2018, and we've taken extra care to make the New Architecture a gradual migration for most apps. In 2021, we created [the New Architecture Working Group](#) to collaborate with the community on ensuring a smooth upgrade experience for the entire React ecosystem.

Most apps will be able to adopt React Native 0.76 with the same level of effort as any other release. The most popular React Native libraries already support the New Architecture. The New Architecture also includes an automatic interoperability layer to enable backward compatibility with libraries targeting the old architecture.

Over the past several years of development, our team has publicly shared our vision for the New Architecture. If you missed any of these talks, check them out here:

- [React Native EU 2019 - The New React Native](#)
- [React Conf 2021 - React 18 Keynote](#)
- [App.js 2022 - Bringing the New React Native Architecture to the OSS Community](#)
- [React Conf 2024 - Day 2 Keynote](#)

What is the New Architecture

The New Architecture is a complete rewrite of the major systems that underpin React Native, including how components are rendered, how JavaScript abstractions communicates with native abstractions, and how work is scheduled across different threads. Although most users should not have to think about how these

systems work, these changes bring improvements and new capabilities.

In the old architecture, React Native communicated with the native platform using an asynchronous bridge. To render a component or call a native function, React Native needed to serialize and enqueue native functions calls with the bridge, which would be processed asynchronously. The benefit of this architecture is that the main thread was never blocked for rendering updates or handling native module function calls, since all work was done on a background thread.

However, users expect immediate feedback to interactions to feel like a native app. This means some updates need to render synchronously in response to user input, potentially interrupting any in-progress render. Since the old architecture was only asynchronous, we needed to rewrite it to allow for both asynchronous and synchronous updates.

Additionally, in the old architecture, serializing function calls over the bridge quickly became a bottleneck, especially for frequent updates or large objects. This made it hard for apps to achieve 60+ FPS reliably. There were also synchronization issues: when the JavaScript and native layer got out of sync, it was impossible to reconcile them synchronously, resulting bugs like lists showing frames of

empty space and visual UI jumps due to intermediate states rendering.

Finally, since the old architecture kept a single copy of the UI using the native hierarchy, and mutated that copy in place, layout could only be computed on a single thread. This made it impossible to process urgent updates like user inputs, and layout could not be read synchronously, such as reading in a layout effect to update the position of a tooltip.

All of these problems meant that it was not possible to properly support React's concurrent features. To solve these problems, the New Architecture includes four main parts:

- The New Native Module System
- The New Renderer
- The Event Loop
- Removing the Bridge

The New Module system allows the React Native Renderer to have synchronous access to the native layer, which allows it to handle events, schedule updates, and read layout both asynchronously and synchronously. The new Native Modules are also lazily loaded by default, giving apps a significant performance gain.

The New Renderer can handle multiple in progress trees across multiple threads, which allows React to process multiple concurrent update priorities, either on the main thread or a

background thread. It also supports reading layout from multiple threads synchronously or asynchronously, to support more responsive UIs without jank.

The new Event Loop can process tasks on the JavaScript thread in a well-defined order. This allows React to interrupt rendering to process events so urgent user events can take priority over lower priority UI transitions. The Event Loop also aligns with web specifications, so we can support for browser features like microtasks, `MutationObserver`, and `IntersectionObserver`.

Finally, removing the bridge allows for faster startup and direct communication between JavaScript and the native runtime, so that the cost of switching work is minimized. This also allows for better error reporting, debugging, and reducing crashes from undefined behavior.

The New Architecture is now ready to be used in production. It is already used at scale at Meta in the Facebook app and in other products. We successfully used React Native and the New Architecture in the Facebook and Instagram app we developed for our [Quest devices](#).

Our partners have already been using the New Architecture in production for months now: have a look at these success stories by [Expensify](#) and [Kraken](#), and give [Bluesky](#) a shot with their new release.

New Native Modules

The new Native Module System is a major rewrite of how JavaScript and the native platform communicate. It's written entirely in C++, which unlocks many new capabilities:

- Synchronous access to and from the native runtime
- Type safety between JavaScript and native code
- Code sharing across platforms
- Lazy module loading by default

In the new Native Module system, JavaScript and the native layer can now synchronously communicate with each other through the JavaScript Interface (JSI), without the need to use an asynchronous bridge. This means your custom Native Modules can now synchronously call a function, return a value, and pass that value back to another Native Module function.

In the old architecture, in order to handle a response from native function calls, you needed to provide a callback, and the value returned needed to be serializable:

```
// ❌ Sync callback from Native
Module
nativeModule.getValue(value => {
  // ❌ value cannot reference a
  native object
  nativeModule.doSomething(value);
});
```

In the New Architecture, you can make synchronous calls to native functions:

```
// ✅ Sync response from Native  
Module  
const value =  
  nativeModule.getValue();  
  
// ✅ value can be a reference to a  
  native object  
  nativeModule.doSomething(value);
```

With the New Architecture, you can finally leverage the full power of a C++ native implementation while still accessing it from JavaScript/TypeScript APIs. The New Module System supports [modules written in C++](#) so you can write your module once, and it works across all platforms, including Android, iOS, Windows, and macOS. Implementing modules in C++ allows for more fine-grained memory management and performance optimizations.

Additionally, with [Codegen](#), your modules can define a strongly typed contract between the JavaScript layer and the native layer. From our experience, cross-boundary type errors are one of the most common sources of crashes in cross-platform apps. Codegen lets you overcome those problems while also generating boilerplate code for you.

Finally, modules are now lazily loaded: they are loaded in memory only when they're effectively needed rather than at startup. This reduces the

app startup time and keeps it low as the application grows in complexity.

Popular libraries such as [react-native-mmkv](#) have already seen benefits from migrating to the new Native Modules:

“The new Native Modules greatly simplified setup, autolinking, and initialization for `react-native-mmkv`. Thanks to the New Architecture, `react-native-mmkv` is now a pure C++ Native Module, which allows it to work on any platform. The new Codegen allows MMKV to be fully type-safe, which fixed a long-standing `NullPointerException` issue by enforcing null-safety, and being able to call Native Module functions synchronously allowed us to replace custom JSI access with the new Native Module API.”

[Marc Rousavy](#), creator of `react-native-mmkv`

New Renderer

We've also completely rewritten the Native Renderer, adding several benefits:

- Updates can be rendered on different threads at different priorities.
- Layout can be read synchronously and across different threads.
- The renderer is written in C++ and shared across all platforms.

The updated Native Renderer now stores the view hierarchy in an immutable tree structure. This means that the UI is stored in a way that cannot be changed directly, allowing for thread-safe processing of updates. This allows it to handle multiple in-progress trees, each representing a different version of the user interface. As a result, updates can be rendered in the background without blocking the UI (such as during transitions) or on the main thread (in response to user input).

By supporting multiple threads, React can interrupt a low-priority update to render an urgent one, such as those generated by user inputs, and then resume the low-priority update as needed. The new renderer can also read layout information synchronously and across different threads. This enables background computation for low-priority updates and synchronous reads when needed, such as repositioning a tooltip.

Finally, rewriting the renderer in C++ allows it to be shared across all platforms. This ensures that the same code runs on iOS, Android, Windows, macOS, and any other React Native-supported platform, providing consistent rendering capabilities without needing re-implementation for each platform.

This is a significant step towards our [Many Platform Vision](#). For example, View Flattening was an Android-only optimisation to avoid deep layout trees. The new renderer, with

shared C++ core, [brings this feature to iOS](#). This optimisation is automatic and does not require setup, it comes for free with the shared renderer.

With these changes, React Native now fully supports Concurrent React features like Suspense and Transitions, making it easier to build complex user interfaces that respond quickly to user input without jank, delays, or visual jumps. In the future, we will leverage these new capabilities to bring more improvements to built-in components such as FlatList and TextInput.

Popular libraries like [Reanimated](#) are already taking advantage of the New Renderer:

“Reanimated 4, currently in development, introduces a new animation engine that works directly with the New Renderer, allowing it to handle animations and manage layout across different threads. The New Renderer’s design is what truly enables these features to be built without relying on numerous workarounds. Moreover, because it’s implemented in C++ and shared across platforms, large portions of Reanimated can be written once, reducing platform-specific issues, minimizing the codebase, and streamlining adoption for out-of-tree platforms.”

[Krzysztof Magiera](#), creator of [Reanimated](#)

The Event Loop

The New Architecture allowed us to implement a well-defined event loop processing model, as described in this [RFC](#). This RFC follows the specifications described in the [HTML Standard](#), and it describes how React Native should perform tasks on the JavaScript thread.

Implementing a well-defined event loop closes gaps between React DOM and React Native: the behavior of a React Native application is now closer to the behavior of a React DOM application, making it easier to learn once, and write anywhere.

The event loop brings many benefits to React Native:

- The ability to interrupt rendering to process events and tasks
- Closer alignment with web specifications
- Foundation for more browser features

With the Event Loop, React is able to predictably order updates and events. This allows React to interrupt a low priority update with an urgent user event, and the New Renderer allows us to render those updates independently.

The Event Loops also aligns the behavior of events and task like timers with web specifications, which means React Native works more like what users are familiar with in

the Web, and allows for better code sharing between React DOM and React Native.

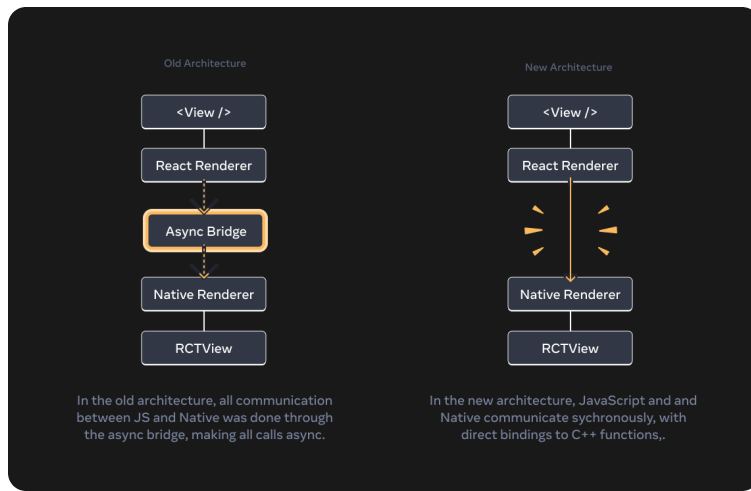
It also allows for the implementation of more compliant browser features like microtasks, `MutationObserver`, and `IntersectionObserver`. These features are not ready to use in React Native yet, but we are working on bringing them to you in the future.

Finally, the Event Loop and the New Renderer changes to support reading layout synchronously allow React Native to add proper support for `useLayoutEffect` to read layout information synchronously and update the UI in the same frame. This allows you to position elements correctly before they are displayed to the user.

See [useLayoutEffect](#) for more details.

Removing the Bridge

In the New Architecture, we've also fully removed React Native's dependency on the bridge, replacing it with direct, efficient communication between JavaScript and native code using JSI:



Removing the bridge improves startup time by avoiding bridge initialization. For example, in the old architecture, in order to provide global methods to JavaScript, we would need to initialize a module in JavaScript on startup, causing a small delay in app startup time:

```
// ❌ Slow initialization
import {NativeTimingModule} from
'NativeTimingModule';
global.setTimeout = timer => {

NativeTimingModule.setTimeout(timer);
};

// App.js
setTimeout(() => {}, 100);
```

In the New Architecture, we can directly bind methods from C++:

```
// ✅ Initialize directly in C++
runtime.global().setProperty(runtime,
"setTimeout", createTimer);
```

```
// App.js
```

```
setTimeout(() => {}, 100);
```

The rewrite also improves error reporting, particularly for JavaScript crashes at startup, and reduces crashes from undefined behavior. If crashes occur, the new [React Native DevTools](#) simplify debugging and support the New Architecture.

The bridge remains for backward compatibility to support gradual migration to the New Architecture. In the future, we will remove the bridge code completely.

Gradual Migration

We expect most apps can upgrade to 0.76 with the same effort as any other release.

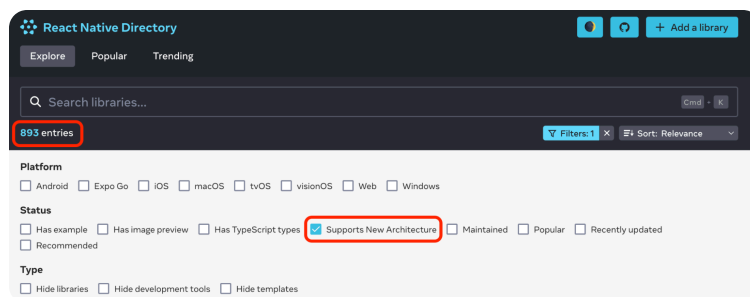
When you upgrade to 0.76, the New Architecture and React 18 are enabled by default. However, to use concurrent features and gain the full benefits of the New Architecture, your app and libraries will need to be gradually migrated to fully support the New Architecture.

When you first upgrade, your app will run on the New Architecture with an automatic interoperability layer with the old architecture. For most apps, this will work without any changes, but there are [known limitations](#) with the interop layer, as it does not support accessing custom Shadow Nodes or concurrent features.

To use concurrent features, apps will also need to be updated to support [Concurrent React](#) by following the [Rules of React](#). To migrate your JavaScript code to React 18 and its semantics, follow the [React 18 Upgrade guide](#).

The overall strategy is to get your application running on the New Architecture without breaking existing code. You can then gradually migrate your app at your own pace. For new surfaces that have migrated all modules to the New Architecture, you can start using concurrent features immediately. For existing surfaces, you may need to address some issues and migrate modules before adding concurrent features.

We've collaborated with the most popular React Native libraries to ensure support for the New Architecture. More than 850 libraries are already compatible, including all libraries with over 200K weekly downloads (~10% of downloaded libraries). You can check library compatibility with the New Architecture on the reactnative.directory website:



For more details on upgrading, see [How to Upgrade](#) below.

New Features

The New Architecture includes full support for React 18, concurrent features, and `useLayoutEffect` in React Native. For a full list of React 18 features, please see the [React 18 blog post](#).

Transitions

Transitions are a new concept in React 18 to distinguish between urgent and non-urgent updates.

- **Urgent updates** reflect direct interaction, like typing and pressing.
- **Transition updates** transition the UI from one view to another.

Urgent updates need immediate response to match our intuitions about how physical objects behave. However, transitions are different because the user doesn't expect to see every intermediate value on screen. In the New Architecture, React Native is able to support rendering urgent updates and transition updates separately.

Typically, for the best user experience, a single user input should result in both an urgent update and a non-urgent one. Similar to ReactDOM, events like `press` or `change` are handled as urgent and rendered immediately. You can use the `startTransition` API inside an input event to inform React which updates

are “transitions” and can be deferred to the background:

```
import {startTransition} from
'react';

// Urgent: Show the slider value
setCount(input);

// Mark any state updates inside as
transitions
startTransition(() => {
  // Transition: Show the results
  setNumberOfTiles(input);
});
```

Separating urgent events from transitions allows for a more responsive user interface, and a more intuitive user experience.

Here's a comparison of the old architecture without transitions and the new architecture with transitions. Imagine that each tile isn't a trivial view with a background color, but a rich component containing images and other components that are expensive to render.

After using `useTransition` you avoid thrashing your app with updates and falling behind.



Before: rendering tiles without marking it as a transition.



After: rendering tiles *with transitions* to interrupt in-progress renders of stale state.

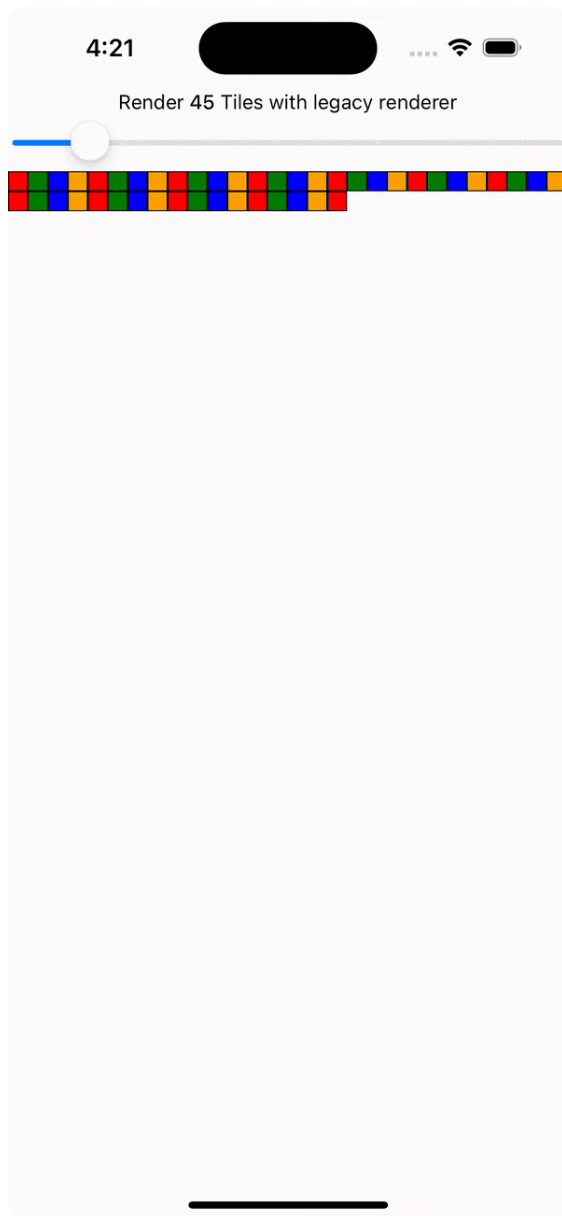
For more information, see [Support for Concurrent Renderer and Features](#).

Automatic Batching

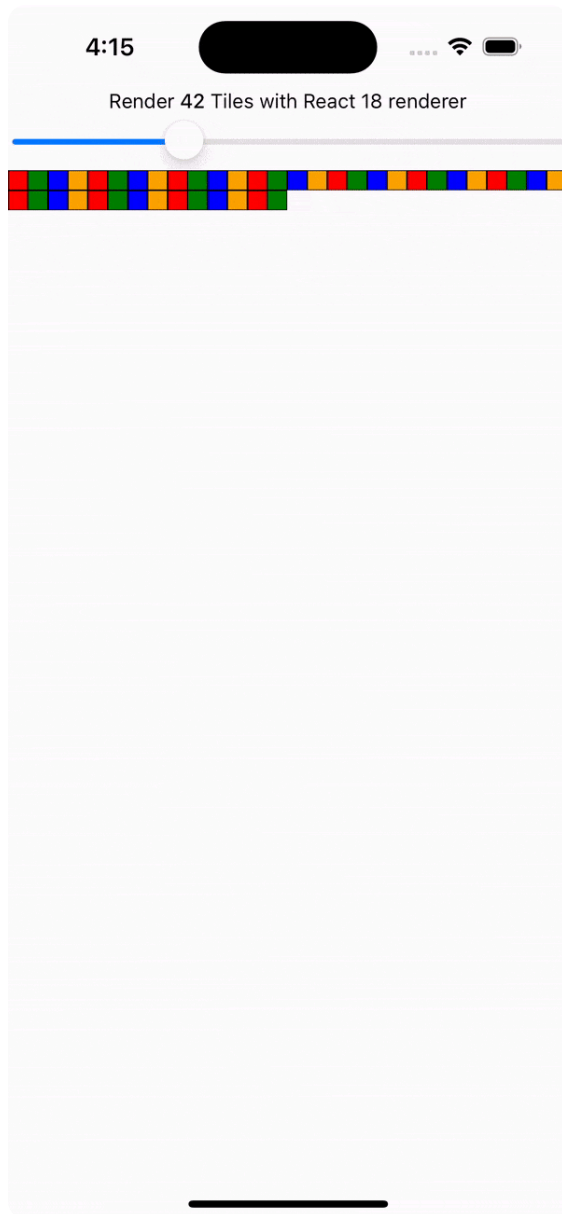
When upgrading to the New Architecture, you will benefit from automatic batching from React 18.

Automatic batching allows React to batch together more state updates when rendering to avoid the rendering of intermediate states. This

allows React Native to be faster and less susceptible to lags, without any additional code from the developer.



Before: rendering frequent state updates with legacy renderer.



After: rendering frequent state updates with *automatic batching*.

In the old architecture, more intermediate states are rendered, and the UI keeps updating even when the slider stops moving. The New Architecture, renders fewer intermediate states and completes the rendering much sooner thanks to automatically batching the updates.

For more information, see [Support for Concurrent Renderer and Features](#).

useLayoutEffect

Building on the Event Loop and the ability to read layout synchronously, in the New Architecture we added proper support for `useLayoutEffect` in React Native.

In the old architecture, you needed to use the asynchronous `onLayout` event to read layout information of a view (which was also asynchronous). As a result there would be at least one frame where the layout was incorrect until the layout was read and updated, causing issues like tooltips placed in the wrong position:

```
// ❌ async onLayout after commit
const onLayout =
  React.useCallback(event => {
    // ❌ async callback to read layout
    ref.current?.measureInWindow((x, y,
      width, height) => {
      setPosition({x, y, width,
        height});
    });
  }, []);

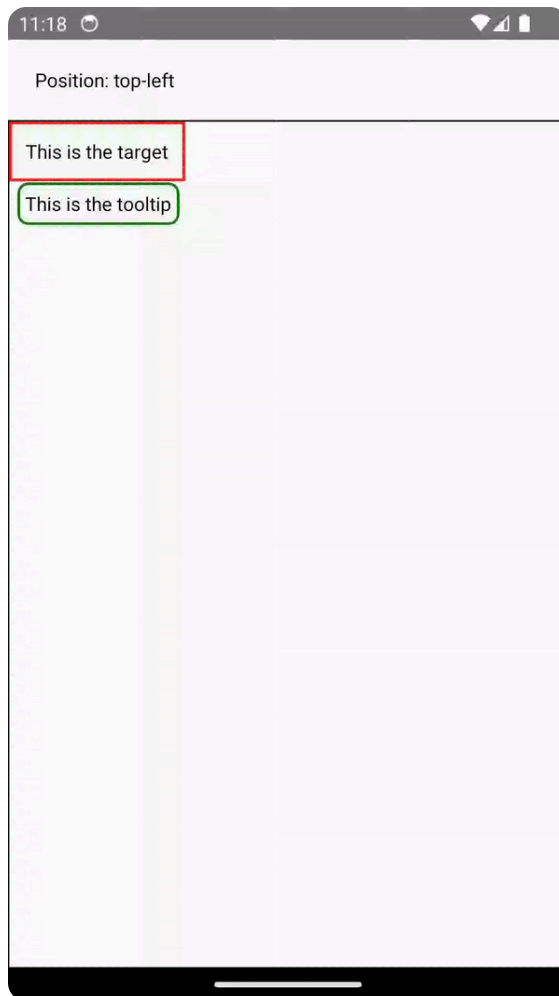
// ...
<ViewWithTooltip
  onLayout={onLayout}
  ref={ref}
  position={position}
/>;
```

The New Architecture fixes this by allowing synchronous access to layout information in

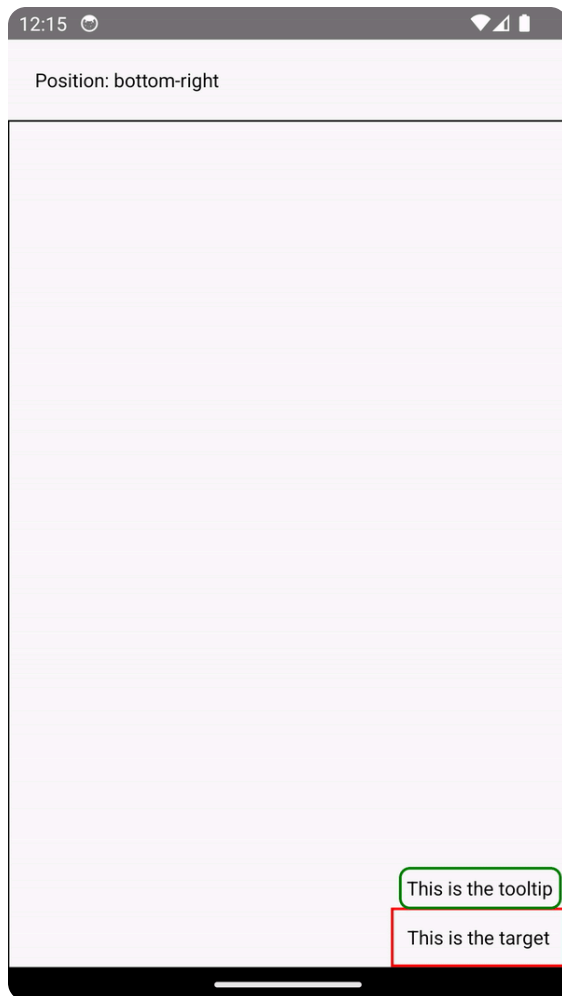
useLayoutEffect:

```
// ✅ sync layout effect during  
commit  
useLayoutEffect(() => {  
  // ✅ sync call to read layout  
  const rect =  
    ref.current?.getBoundingClientRect();  
  setPosition(rect);  
}, []);  
  
// ...  
<ViewWithTooltip ref={ref} position=  
  {position} />;
```

This change allows you to read layout information synchronously and update the UI in the same frame, allowing you to position elements correctly before they are displayed to the user:



In the old architecture, layout was read asynchronously in `onLayout`, causing the position of the tooltip to be delayed.



In the New Architecture, layout can be read in `useLayoutEffect` synchronously, updating the tooltip position before displaying.

For more information, see the docs for [Synchronous Layout and Effects](#).

Full Support for Suspense

Suspense lets you declaratively specify the loading state for a part of the component tree if it's not yet ready to be displayed:

```
<Suspense fallback={<Spinner />}>  
  <Comments />  
</Suspense>
```

```
</Suspense>
```

We introduced a limited version of Suspense several years ago, and React 18 added full support. Until now, React Native was not able to support concurrent rendering for Suspense.

The New Architecture includes full support for Suspense introduced in React 18. This means that you can now use Suspense in React Native to handle loading states for your components, and the suspended content will render in the background while the loading state is displayed, giving higher priority to user input on visible content.

For more, see the [RFC for Suspense in React 18](#).

How to Upgrade

To upgrade to 0.76, follow the steps in the [release post](#). Since this release also upgrades to React 18, you will also need to follow the [React 18 Upgrade guide](#).

These steps should be enough for most apps to upgrade to the New Architecture thanks to the interop layer with the old architecture.

However, to take full advantage of the New Architecture and to start using concurrent features, you will need to migrate your custom Native Modules and Native Components to

support the new Native Module and Native Component APIs.

Without migrating your custom Native Modules, you will not get the benefits of shared C++, synchronous method calls, or type-safety from codegen. Without migrating your Native Components, you will not be able to use concurrent features. We recommend migrating all Native Components and Native Modules to the New Architecture as soon as possible.

NOTE

In a future release, we will remove the interop layer and modules will need to support the New Architecture.

Apps

If you are an app developer, to fully support the New Architecture, you will need to upgrade your libraries, custom Native Components, and custom Native Modules to fully support the New Architecture.

We've collaborated with the most popular React Native libraries to ensure support for the New Architecture. You can check library compatibility with the New Architecture on the reactnative.directory website.

If any of the libraries your app depends on are not compatible yet, you can:

- Open an issue with the library and ask the author to migrate to the New Architecture.
- If the library is not maintained, consider alternative libraries with the same features.
- [Opt-out from the New Architecture](#) while those libraries are migrated.

If your app has custom Native Modules or custom Native Components, we expect them to work fine, thanks to our [interop layer](#).

However, we recommend upgrading them to the new Native Module and Native Component APIs to fully support the New Architecture and adopt concurrent features.

Please follow these guides to migrate your modules and components to the New Architecture:

- [Native Modules](#)
- [Native Components](#)

Libraries

If you are a library maintainer, please first test that your library works with the interop layer. If it does not, please open an issue on the [New Architecture Working Group](#).

To fully support the New Architecture, we recommend migrating your library to the new Native Module and Native Component APIs as soon as possible. This will allow users of your

library to take full advantage of the New Architecture and support concurrent features.

You can follow these guides to migrate your modules and components to the New Architecture:

- [Native Modules](#)
- [Native Components](#)

Opt-out

If, for any reason, the New Architecture is not behaving properly in your application, there is always the option to opt-out from it until you are ready to turn it on again.

To opt-out from the New Architecture:

- On Android, modify the `android/gradle.properties` file and turn off the `newArchEnabled` flag:

```
-newArchEnabled=true  
+newArchEnabled=false
```

- On iOS, you can reinstall the dependencies by running the command:

```
RCT_NEW_ARCH_ENABLED=0 bundle exec  
pod install
```

Thanks

Delivering the New Architecture to the OSS community has been a huge effort that took us several years of research and development. We want to take a moment to thank all the current and past members of the React team who helped us achieve this result.

We are also extremely grateful to all the partners who collaborated with us to make this happen. Specifically, we would like to call out:

- [Expo](#), for adopting the New Architecture early on, and for supporting the work on migrating the most popular libraries.
- [Software Mansion](#), for maintaining crucial libraries in the ecosystem, for migrating them to the New Architecture early and for all the help in investigating and fixing various issues.
- [Callstack](#), for maintaining crucial libraries in the ecosystem, for migrating them to the New Architecture early and for the support with the work on the Community CLI.
- [Microsoft](#), for adding the New Architecture implementation for `react-native-windows` and `react-native-macos` as well as in several other developer tools.
- [Expensify](#), [Kraken](#), [Bluesky](#) and [Brigad](#) for pioneering the adoption of the New Architecture and reporting various issues so that we could fix them for everyone else.
- All the independent library maintainers and developers who contributed to the New Architecture by testing it, fixing some of

the issues, and opening questions on unclear matters so that we could clear them.

Tags:

announcement